
Tentamen 2023-05-05
Programspråk (5DV086)

– H. Björklund –

Totalt antal uppgifter	7
Maximalt antal poäng	60
Krav för betyg 3	36

För godkänt på tentamen krävs 60%, dvs 36 poäng. De resterande 24 poängen kommer att viktas och slås ihop med poängen från inlämningsuppgifterna för att avgöra om betyget kan bli 4 eller 5.

Kom ihåg att vi bara kan rätta och ge poäng till svar vi kan läsa. *Skriv tydligt!*

Problem 1

10 poäng

Låt L vara språket över alfabetet $\{a, b\}$ definierat av

$$L = \{a^n b^m \mid n \neq 2m\}.$$

L består med andra ord av alla strängar som består först av en sekvens av instanser av a och sedan en sekvens av instanser av b sådana att antalet a inte är exakt hälften av antalet b .

Konstruera en kontextfri grammatik som genererar L .

Problem 2

10 poäng

Låt L_1 vara det oändliga strängspråket över alfabetet $\{a, b\}$ som har följande egenskaper:

Om s är en sträng i L_1 så

- I. innehåller s exakt lika många a som b
- II. innehåller varje prefix¹ av s minst lika många a som b

L_1 består alltså av alla strängar som uppfyller I. och II., men inga andra strängar.

Exempel på strängar i L_1 :

- ε
- ab
- $aabb$
- $aababbab$

Konstruera en kontextfri grammatik som genererar språket L_1 .

Problem 3

6 poäng

Beskriv kortfattat begreppen precedens och associativitet. Hur är de relaterade i kontexten programspråk?

Problem 4

8 poäng

En interpretator består normalt sett av en Scanner/Lexer, en Parser, och en del som exekverar koden. Vilka är deras uppgifter och vilka typer av datastrukturer använder sig de typiskt av?

¹Ett *prefix* av en sträng s är en sträng t som man får från s genom att börja läsa från början av s och sedan sluta någonstans, inte nödvändigtvis först där s tar slut. Strängen $abca$ har alltså prefixen ε, a, ab, abc , och $abca$.

Problem 5**6 poäng**

I rent funktionella språk säger vi att det inte finns några variabler. Ändå kan vi i både funktionella och imperativa språk se uttryck som

$$a = b + 4$$

(med viss variation i den exakta syntaxen). I imperativa språk kallar vi det för en variabeltilldelning medan vi i funktionella språk kallar det för att namnge ett värde (eller binda ett värde till ett namn). Vad beror det på? Vad är skillnaden?

Problem 6**10 poäng**

Betrakta följande två Haskell-funktioner:

```
foo x y = filter (\w -> x == w) y
```

```
bar x y = foldl (\w -> \q -> (if q /= x then w+1 else w)) 0 y
```

Beskriv kortfattat vad respektive funktion gör.

Vad blir resultatet av följande anrop?

```
foo 'c' "aabcccaacbb"
```

```
bar 'c' "aabcccaacbb"
```

Problem 7**10 poäng**

Skriv en funktion `myRev` i Haskell som vänder på listor:

```
myRev "Hello"
```

```
"olleH"
```

```
myRev [1, 2, 3]
```

```
[3, 2, 1]
```

Din funktion **får inte** använda sig av inbyggda funktioner som “rev”, “map”, “foldl”, “filter” etc. **Din funktion ska ha en tidskomplexitet som är linjär i listans längd**, det vill säga $\mathcal{O}(n)$, där n är listans längd.